

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto: GS Stock & Facturación.

Link del documento de software:

Genser Catalán 23401, Fernando Ruíz 23065, Hugo Barillas 23306, Jose López 23773

Link al documento: [Click documento](#)

Link de la presentación: [Click presentacion](#)

Link video presentación: [Click para video](#)

Link repositorio GITHUB: [Ir a github](#)

Erick Marroquín

Ingeniería en Software 1 – CC3090

Guatemala, 2025

Product Backlog

GSS-157 Migración Línea de zapatos
GSS-158 Actualización Inventario (Línea de producto en endpoints)
GSS-159 API Post inventario (crear producto)
GSS-160 API Get Inventario (lista productos)
GSS-161 API Post (actualizar productos)
GSS-162 API Delete (inactivar productos)
GSS-173 Responsive
GSS-177 UI “Pagos y Devoluciones”
GSS-178 Formulario devoluciones
GSS-179 Formulario registrar pago
GSS-180 Tabla historial de pagos
GSS-181 Filtro clientes y fechas
GSS-182 Tabla historial de devoluciones
GSS-183 Filtro clientes y fechas
GSS-184 Middleware para “Pagos y Devoluciones”
GSS-185 Agregar columna “Línea de producto” en la tabla
GSS-186 Eliminar modal del login
GSS-187 Migración para incluir línea nacional

GSS-188 Migración para incluir línea importadora
GSS-189 Modificar botón eliminar → desactivar
GSS-190 Modificar campo teléfono en clientes
GSS-192 Actualizar header “Pagos y Devoluciones”
GSS-194 Test
GSS-195 Test Frontend
GSS-196 Test Frontend
GSS-197 Test Frontend
GSS-198 Test Backend
GSS-199 Test Backend
GSS-200 Test Backend

Finalizadas

Pila del Sprint

GSS ID	Nombre de la tarea	Puntos de Historia
--------	--------------------	--------------------

GSS-1 57	1. Tarea Principal: Migración Línea de zapatos	3
GSS-1 87	1.1 Subtarea: Migración para incluir línea nacional	1
GSS-1 88	1.2 Subtarea: Migración para incluir línea importadora	1
GSS-1 58	2. Tarea Principal: Actualización Inventario (Para incluir Línea de producto en los endpoints)	3
GSS-1 59	2.1 Subtarea: Actualizar API Post inventario (Para crear nuevo producto)	1
GSS-1 60	2.2 Subtarea: Actualizar API Get Inventario (Para devolver lista de productos)	1
GSS-1 61	2.3 Subtarea: Actualizar API Post (Para actualizar productos)	1
GSS-1 62	2.4 Subtarea: Actualizar API Delete (Para inactivar productos)	1
GSS-1 85	4.3 Subtarea: Agregar columna “Línea de producto” en la tabla	1
GSS-1 73	4.4 Subtarea: Responsive	1
GSS-1 77	6. Tarea Principal: UI “Pagos y Devoluciones”	3
GSS-1 78	6.1 Subtarea: Formulario para devoluciones con los campos: Vendedor, Cliente, Línea del producto, Código, Talla, Cantidad, Cuadro de texto	1

GSS-1 79	6.2 Subtarea: Formulario para registrar pago con los campos: Vendedor, Cliente, Línea del producto, Método de pago, Monto pagado, Cuadro de texto	1
GSS-1 80	6.3 Subtarea: Tabla con historial de pagos	1
GSS-1 81	6.4 Subtarea: Filtro clientes y fechas	1
GSS-1 82	6.5 Subtarea: Tabla con historial de devoluciones	1
GSS-1 83	6.6 Subtarea: Filtro clientes y fechas	1
GSS-1 92	6.7 Subtarea: Actualizar header para incluir “Pagos y Devoluciones”	1
GSS-1 84	7. Tarea Principal: Agregar Middleware para pantalla “Pagos y Devoluciones” (roles: Vendedor, Administrador)	2
GSS-1 86	8. Tarea Principal: Eliminar modal del login	2
GSS-1 89	9. Tarea Principal: Modificar función del botón “eliminar” para “desactivar” en pantalla gestión usuarios	2
GSS-1 90	10. Tarea Principal: Modificar campo número de teléfono en gestión de clientes (Aceptar + (0-9))	2
GSS-1 94	11. Tarea Principal: Test	2
GSS-1 95	11.1 Subtarea: Test Frontend	1

GSS-1 96	11.2 Subtarea: Test Frontend	1
GSS-1 97	11.3 Subtarea: Test Frontend	1
GSS-1 98	11.4 Subtarea: Test Backend	1
GSS-1 99	11.5 Subtarea: Test Backend	1
GSS-2 00	11.6 Subtarea: Test Backend	1

Calendario de planificación

Fecha	Tarea
11/08/2025	GSS-157 Migración Línea de zapatos
12/08/2025	GSS-187 Migración para incluir línea nacional
13/08/2025	GSS-188 Migración para incluir línea importadora
14/08/2025	GSS-158 Actualización Inventario (Línea de producto en endpoints)
15/08/2025	GSS-159 API Post inventario (crear producto)
15/08/2025	GSS-160 API Get Inventario (lista productos)
16/08/2025	GSS-161 API Post (actualizar productos)
16/08/2025	GSS-162 API Delete (inactivar productos)
17/08/2025	GSS-185 Agregar columna “Línea de producto” en la tabla
17/08/2025	GSS-173 Responsive
18/08/2025	GSS-177 UI “Pagos y Devoluciones”

18/08/2025	GSS-178 Formulario devoluciones
18/08/2025	GSS-179 Formulario registrar pago
19/08/2025	GSS-180 Tabla historial de pagos
19/08/2025	GSS-181 Filtro clientes y fechas
20/08/2025	GSS-182 Tabla historial de devoluciones
20/08/2025	GSS-183 Filtro clientes y fechas
21/08/2025	GSS-192 Actualizar header “Pagos y Devoluciones”
21/08/2025	GSS-184 Middleware para “Pagos y Devoluciones”
22/08/2025	GSS-186 Eliminar modal del login
22/08/2025	GSS-189 Modificar botón eliminar → desactivar
23/08/2025	GSS-190 Modificar campo teléfono en clientes
24/08/2025	GSS-194 Test Principal
24/08/2025	GSS-195 Test Frontend
25/08/2025	GSS-196 Test Frontend
25/08/2025	GSS-197 Test Frontend
26/08/2025	GSS-198 Test Backend
26/08/2025	GSS-199 Test Backend
27/08/2025	GSS-200 Test Backend

Incremento

Link al repositorio: <https://github.com/JosFer720/GS-Stock-y-Facturacion-Frontend>

Estimación de Tiempo y Costo del desarrollo

Metodo Function Points

1. Cálculo (PCU)

$$PCU = FPA + FPCU$$

$$PCU = 5 + 85 = 90$$

2. Cálculo (FPA)

Peso de actores	Complejidad
Jefe	3
Secretaria	1
Vendedor	1
Total	5

3. Cálculo (FPCU)

Peso caso de usos	Peso
Gestión de inventario	15
Facturación	15
Ventas	15
Autenticación	10
Gestion clientes	10
Reportes	10
Consultas básicas	5
Alertas de stock	5

$$FPCU = (15 \times 3) + (10 \times 3) + (5 \times 2) = 85$$

4. Cálculo (PCUA)

$$PCUA = PCU \times FCT \times FA$$

$$PCUA = 90 \times 0.89 \times 0.74 = 59$$

5. Cálculo (FCT)

Factor	Valoración
Sistema distribuido (Docker, microservicios)	4
Rendimiento (gestión de inventario en tiempo real)	4
Seguridad (JWT, roles)	5
Transacción de ventas.	4
Entrenamiento a usuarios	2
Reusabilidad de código	3
Portabilidad	3
Transacciones ACID (ventas, inventario)	4

$$FCT = 0.6 + (0.01 \times \sum 29) = 0.89$$

6. Cálculo (FA)

Factor	Valoración
Familiaridad con el modelo de proyecto	4
Experiencia en orientación a objetos	5
Capacidad del lider	4
Motivación	4
Estabilidad de requerimientos	3

Dificultad del lenguaje	2
	22

$$FA = 1.4 + (-0.03 \times 22) = 0.74$$

7. Cálculo (E)

$$E = PCUA \times FC$$

$$E = 59 \times 20 = 1180h$$

8. Calculo costo estimado

$$\text{Costo total} = 1180h \times \$20 = \$23\,600 = Q181\,194$$

Ejecución del presupuesto

Horas trabajadas por día:

$$4 \text{ devs} \times 1.5 \text{ h/día} = 6 \text{ h/día}$$

Horas trabajadas por mes:

$$6 \text{ h/día} \times 30 \text{ días} \approx 180 \text{ h/mes}$$

Horas acumuladas en 5 meses:

$$180 \text{ h/mes} \times 5 \text{ meses} = 900$$

Gasto en dólares

$$900 \text{ h} \times 15 \text{ USD/h} = 13,500 \text{ USD}$$

Convertir a quetzales con tipo de cambio es Q7.8/USD:

$$13,500 \text{ USD} \times 7.8 = Q105,300$$

Porcentaje del presupuesto gastado

$(Q105,300 / Q181,194) \times 100 \approx 58.1\%$

porcentaje de tiempo consumido

$(5 \text{ meses} / 7.5 \text{ meses}) \times 100 \approx 66.7\%$

Método utilizado

Para estimar el esfuerzo de desarrollo, utilizamos la técnica de Puntos de Casos de Uso (PCU), ya que nos pareció una forma objetiva y estructurada de calcular el trabajo necesario. Primero identificamos y clasificamos a los actores del sistema para calcular el Factor de Peso de Actores (FPA), y luego analizamos la complejidad de cada caso de uso para determinar el Factor de Peso de Casos de Uso (FPCU).

Sumando ambos, obtuvimos los PCU brutos. Después, ajustamos esa cifra teniendo en cuenta factores técnicos como requisitos de rendimiento o seguridad (FCT), así como factores del entorno como la experiencia del equipo o qué tan estables eran los requerimientos (FA). Finalmente, multiplicamos los Puntos de Casos de Uso Ajustados (PCUA) por una productividad estándar, en nuestro caso, usamos 20 horas por PCUA para determinar el esfuerzo total.

Costo de tiempo y desarrollo

El resultado que obtuvimos fue un estimado de 1,180 horas de trabajo, lo que se traduce en unos 7.4 meses de desarrollo si asumimos una jornada de 160 horas al mes. En cuanto al costo, calculamos que serían unos \$23,600 dólares, considerando una tarifa promedio de \$20 por hora. Convertido a moneda local, eso representa aproximadamente Q181,194, usando un tipo de cambio de 7.68 quetzales por dólar.

Este cálculo incluye el desarrollo base, más los ajustes por complejidad técnica y factores ambientales, aunque no contempla posibles gastos imprevistos

Prueba

Investigación de tecnologías

Frontend:

Para las pruebas del frontend, evaluamos varias tecnologías, principalmente Cypress y Vitest. Cypress destaca por permitir probar cómo los usuarios interactúan con la aplicación en un navegador, mostrando paso a paso la ejecución de las pruebas y ayudando a identificar errores de manera visual. Además, permite probar flujos completos de la aplicación y gestionar datos de prueba de forma sencilla.

Vitest, por su parte, es más reciente pero muy rápido y fácil de integrar, especialmente en proyectos contruidos con Vite. También permite probar componentes y la interacción con el DOM, lo que lo hace interesante para analizar la calidad del frontend. Comparando estas herramientas, queríamos entender cuál nos ofrecería una mejor combinación de velocidad, facilidad de uso y cobertura de pruebas para nuestro proyecto.

Backend:

En el backend, consideramos herramientas como Jest y Vitest para evaluar su capacidad de probar la lógica del backend en las APIs. Jest es una opción madura y ampliamente usada, con soporte para simulaciones, pruebas paralelas y documentación amplia, lo que facilita revisar funciones críticas en las varias funcionalidades del proyecto.

Vitest, aunque más reciente, ofrece ejecución rápida y flexibilidad, y también permite probar funciones de backend y APIs con ESM y fixtures. Analizamos estas tecnologías para ver cuál nos permitiría detectar errores de manera eficiente y mantener el backend estable mientras desarrollamos nuevas funcionalidades.

Tecnologías elegidas

Después de analizar varias opciones para las pruebas automatizadas, decidimos usar Vitest para el frontend y SuperJest para el backend. Elegimos Vitest porque se integra muy bien con Vue, que es la tecnología principal de nuestro proyecto, y permite realizar pruebas rápidas de componentes e integración de manera sencilla.

Para el backend optamos por SuperJest, ya que se adapta perfectamente a Express, nuestro framework de servidor, y ofrece funciones confiables para probar APIs y la lógica del servidor. En ambos casos, la compatibilidad con las tecnologías que estamos usando fue el factor principal para tomar la decisión, asegurando que las pruebas automatizadas se puedan implementar de forma eficiente y sin problemas de integración.

Videos de pruebas

Pruebas Frontend

1. HeaderComponent - Prueba funcional

Este test monta el header con un router en memoria y verifica el estado visual del enlace activo. Primero comprueba que al iniciar en /dashboard el link correspondiente se marca como activo; luego navega a /productos y valida que el activo cambie a ese enlace y que /dashboard deje de estar activo.

Video: <https://youtu.be/xK2fEHuZCHU>

2. TallasModal – Prueba funcional

Este test valida el comportamiento del modal de tallas: cuando show=true renderiza la lista con los datos correctos; al hacer clic en los controles de cierre emite el evento close; y cuando show=false no muestra contenido.

Video: <https://youtu.be/VYG4fe4l1gE>

3. Tabla Productos – Prueba de integración

Esta prueba se encarga de comprobar que la tabla de productos funcione bien, mostrando los datos como debe, aplicando los estilos según el stock y comunicándose correctamente con su componente principal. También revisa que las acciones del usuario, como seleccionar productos o activar el modo de eliminación, se realicen sin problemas

Video: <https://youtu.be/RDiZWzypUKc>

Pruebas Backend

1. Cliente test – Smoke test

Este smoke test examina que la ruta /clientes de tu aplicación Express esté accesible y devuelva una respuesta en formato JSON, con un código de estado válido (ya sea 200 OK si todo funciona, o 500 en caso de error interno). Para lograrlo, se simula el middleware de autenticación y la conexión a la base de datos con mocks, de modo que la prueba no depende de servicios externos

Video: [Smoke test cliente / inventario](#)

2. Inventario test – Smoke test

verifica que la ruta /inventory del backend funcione en condiciones mínimas sin depender de la base de datos real, del middleware de roles ni de los servicios de sockets (todos se reemplazan por mocks). En concreto, examina que al hacer un GET /inventory la aplicación responda con un código 200, un objeto JSON que contenga la propiedad success: true y un campo data que sea un arreglo. Con esto se asegura que la ruta existe, responde correctamente y mantiene el formato esperado, dejando para pruebas más profundas la lógica de negocio o conexión real a la base de datos

Video: [Smoke test cliente / inventario](#)

3. Auth test – smoke test

Envía peticiones a los endpoints de autenticación (/login, /logout, /forgot-password) y confirmar que devuelven algún código de estado válido. No se valida la lógica completa ni los datos de la base, solo que el servidor arranque correctamente y los endpoints contesten

Video: [Auth smoke test](#)

Encuesta NPS

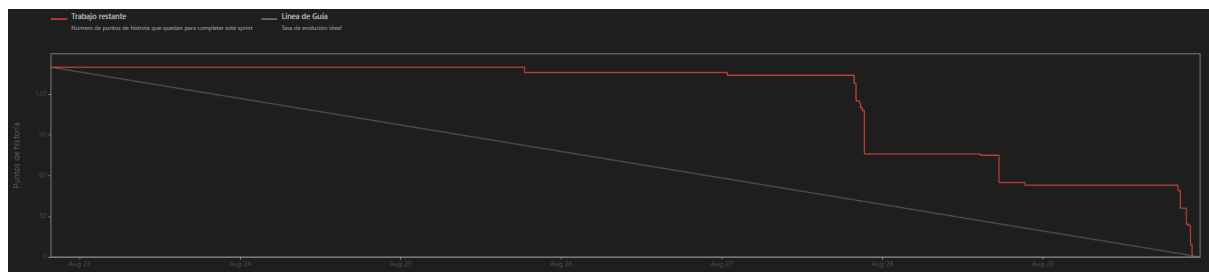
[Click para ir a la encuesta](#)

Revisión técnica

[Revisión técnica](#)

Resultados del Sprint

Gráfico Burndown

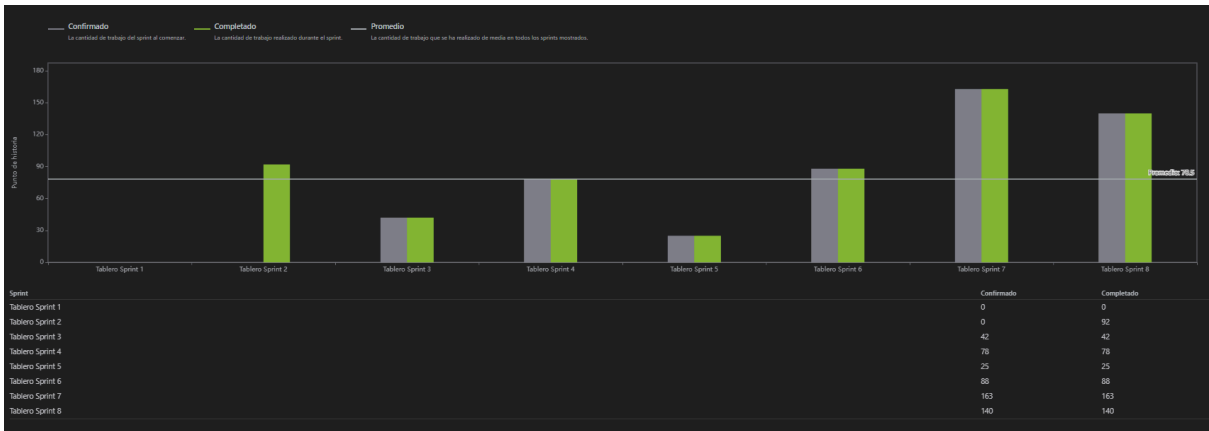


Analizando el gráfico burndown podemos observar que el inicio del trabajo fue lento y con poca constancia, lo que provocó que durante los primeros días el avance no fuera tan notorio como se esperaba. Además, se identifican ciertos espacios muertos entre la entrega de algunas tareas, lo que indica que el equipo no mantuvo un ritmo de trabajo uniforme a lo largo del sprint. Ya en la parte final, más cerca de la fecha de entrega, se percibe un cambio en la dinámica, donde el equipo adoptó un flujo de trabajo más continuo y acelerado, reflejado en los descensos bruscos de la gráfica. Estos descensos muestran que se concentró un gran número de avances en un periodo corto, como respuesta a la presión del tiempo restante.

A pesar de los espacios muertos y de los bajones bruscos que se aprecian en la gráfica, todas las tareas fueron concluidas dentro del plazo de tiempo establecido, lo que demuestra

compromiso y capacidad de respuesta por parte del equipo. Sin embargo, el hecho de haber cumplido con el objetivo no significa que la gestión del trabajo haya sido la más adecuada. Para futuros sprint sería recomendable una mejor organización para mantener un mejor flujo de trabajo.

Métrica de velocidad



Analizando la métrica de velocidad para este sprint, podemos observar que se ha mantenido un constante compromiso del equipo con el objetivo de cumplir todas las tareas establecidas, lo que refleja claramente la responsabilidad y dedicación hacia el proyecto. Si bien el flujo de trabajo no ha sido el más óptimo, como se evidencia en otras gráficas como la burndown, la velocidad muestra una consistencia en la entrega de resultados, garantizando que las tareas fueran completadas dentro del tiempo previsto.

Esto indica que, aunque la organización interna y la distribución del esfuerzo podrían mejorar, el equipo cuenta con la capacidad y disciplina necesarias para responder a los objetivos planteados, asegurando que los entregables se mantengan dentro de los plazos acordados y reforzando así la confianza en su rendimiento.

Indicador numérico del éxito del sprint

7/10

Discusión del éxito del sprint

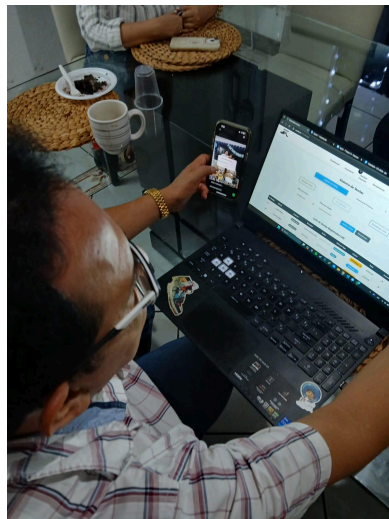
El Sprint 7 cerró con un resultado positivo en cuanto a cumplimiento, completando el 100% de los 140 puntos de historia planificados. Esto se refleja en la gráfica de velocidad, donde el

sprint alcanzó uno de los valores más altos registrados hasta el momento. Este aumento muestra que la capacidad del equipo se encuentra en expansión y que fue posible manejar un volumen mayor de trabajo con éxito.

No obstante, el gráfico burndown revela que el flujo de avance no fue constante. Durante los primeros días apenas se evidenció reducción de tareas, acumulando pendientes que solo comenzaron a resolverse en la parte final. Esto generó descensos abruptos cerca de la fecha de cierre, lo que indica que la mayoría de los entregables se completaron bajo presión de tiempo.

Si bien el resultado final fue satisfactorio y el Product Owner expresó conformidad con los incrementos entregados, la falta de constancia en la gestión del flujo limita el puntaje del sprint. Por ello, la calificación se establece en 7/10, reconociendo la entrega total de objetivos pero destacando la necesidad de mayor disciplina en la administración del tiempo.

Evidencia de muestra del incremento desarrollado a usuarios finales



Para este sprint se presentaron al product owner los avances relacionados con el nuevo requerimiento solicitado, que consistió en la incorporación de dos nuevas líneas de producto: la línea de producto nacional y la línea de producto importado. Durante la demostración, el cliente expresó su satisfacción, ya que destacó que estas mejoras le facilitarán el trabajo diario y le ahorrarán tiempo en la gestión de operaciones.

En particular, valoró mucho la automatización del apartado de "generación de envíos", una tarea que anteriormente debía completar manualmente con los datos de los distintos clientes y que ahora se realiza de forma automática, reduciendo significativamente el esfuerzo y la posibilidad de errores.

Además, también se trabajó en la pantalla de "pagos y devoluciones", donde se podrán registrar los pagos realizados por los clientes a sus cuentas por cobrar y se manejó la lógica

necesaria para los casos en que los clientes devuelven producto, incluyendo la actualización del inventario y los movimientos correspondientes en la cuenta por cobrar.

El product owner se mostró bastante complacido con los avances logrados y, de cara al siguiente sprint, sugirió algunas mejoras adicionales en la plantilla de envíos, como la incorporación del logo de la empresa, con el objetivo de hacer los documentos más personalizados a la empresa.

Retrospectiva del sprint

En este sprint se concretaron funcionalidades clave como la migración de líneas de producto, la automatización de envíos y la pantalla de pagos y devoluciones. Estas tareas aportaron un valor tangible al cliente, quien reconoció que ahora los procesos son más ágiles y menos propensos a errores manuales.

La retrospectiva muestra, sin embargo, que el trabajo estuvo marcado por picos de cierre en lugar de un avance progresivo. La gráfica burndown confirma que gran parte del esfuerzo se concentró en los últimos días, generando un ritmo irregular y mayor presión sobre el equipo. Aunque se cumplió el compromiso total, esta práctica no es sostenible a largo plazo y aumenta el riesgo de errores.

El equipo identificó que la principal mejora pendiente está en la gestión del tiempo y la distribución del esfuerzo. Para próximos sprints se plantea reforzar la planificación diaria, dividir las tareas en entregas parciales y mantener revisiones intermedias para evitar acumulación hacia el final. Esto permitirá no solo cumplir con lo planificado, sino hacerlo de manera más equilibrada y sostenible.

Sistema de control:

- [Link al jira](#)

Excel con control de tiempos

- [Horario software.xlsx](#)

Anexo

Backend:

Cliente test – Smoke test


```
PASS tests/pedido.test.js
PASS tests/cliente.test.js
```

Inventario test – Smoke test

```
PASS tests/pedido.test.js
PASS tests/cliente.test.js
```

Auth test – smoke test

```
PASS tests/usuarios.test.js
PASS tests/auth.test.js
```

Frontend:

HeaderComponent - Prueba funcional

```
✓ src/tests/unit/HeaderComponent.active.test.js
  ✓ HeaderComponent (active link) > marca de
  ✓ HeaderComponent (active link) > actuali

Test Files  1 passed (1)
Tests       2 passed (2)
Start at    17:45:21
Duration    1.46s (transform 74ms, setup 0s)
```

TallasModal – Prueba funcional

```
✓ TallasModal > render
✓ TallasModal > emite "
✓ TallasModal > no rend

Test Files  1 passed (1)
Tests       3 passed (3)
Start at    17:46:37
Duration    1.51s (transf
```

Tabla Productos – Prueba de integración

```
Test Plan: 3 passed (1)
Tests: 7 passed (7)
Start at: 04:46:36
Duration: 1.13s (transfer time, some
[exit] waiting for file changes...
press h to show help, press q to q
```